

Formal Verification of Functional Correctness for the OpenHarmony LiteOS-M Kernel

Tianqi Zhao¹, Qinxiang Cao³, Shenghua Feng¹, Minghui Zhou^{1,2}(✉),
Naijun Zhan^{1,2}, Yongzhi Cao^{1,2}, Junfeng Zhao^{1,2}, Haiyan Zhao^{1,2},
Hao Wang⁴, and Zhenjiang Hu^{1,2}

¹ Zhongguancun Laboratory, Beijing, China

² Peking University, Beijing, China

³ Shanghai Jiao Tong University, Shanghai, China

⁴ Shenzhen Kaihong Digital Industry Development Co., Ltd., Shenzhen, China

Abstract. OpenHarmony LiteOS-M, a preemptive operating system (OS) kernel for the Internet of Things (IoT), is widely deployed in safety-critical domains, such as aerospace and transportation. As a rigorous method to assure software safety, formal verification has been applied to OS kernels in industry. However, entirely verified kernels with large code-bases are rare, since such verification is typically performed within interactive theorem provers, requiring substantial human effort. In this paper, we present the *functional correctness* verification of LiteOS-M. First, to improve verification efficiency, we design a formal verification platform, Smart Verifier. The platform employs an annotation-based verifier as the front end, while the back end integrates Z3 and Rocq, combining automatic and interactive theorem proving techniques. Second, we tailor two verification methods, expressing program refinement as standard Hoare logic triples and modeling concurrency through state transition systems, to utilize the platform for verifying LiteOS-M. Our verified LiteOS-M kernel consists of 17,000 lines of C. During the code review and verification, we find a total of 17 bugs, all confirmed and fixed by developers.

Keywords: OS kernel · Formal Verification · LiteOS-M · Separation Logic

1 Introduction

In the era of all-scenarios, all-connectivity, and all-intelligence, OpenHarmony LiteOS-M [7]—a preemptive OS kernel originally developed by Huawei for IoT in 2020 and presently maintained by the open-source OpenHarmony community—stands out for its small size, low consumption, and high performance. Due to its lightweight nature, the LiteOS-M kernel (referred to as LiteOS-M for short) is widely adopted in microcontroller-based devices with constrained hardware resources. To date, LiteOS-M has been deployed by over 111 vendors spanning industries like aerospace, industrial control, and transportation, and is powering millions of IoT devices globally. Owing to its application in safety-critical

domains, even a minor error or vulnerability in LiteOS-M may cause significant failures of the entire software system, potentially resulting in substantial economic losses or posing a threat to human safety. Therefore, ensuring the correctness of LiteOS-M is of paramount importance.

Formal verification, which leverages mathematical proof techniques to establish properties about programs, is a crucial approach to guaranteeing the correctness of system software. Our team is dedicated to advancing research in formal verification. Motivated by the industry’s urgent demand and our interest in exploring the practical application of formal verification techniques, we have collaborated with an enterprise focused on the OpenHarmony ecosystem to verify the *functional correctness* of LiteOS-M. Specifically, our goal is to ensure that the kernel’s C implementation is free from implementation defects.

It is feasible to formally prove the functional correctness of OS kernels, as shown by notable projects including seL4 [5], Verisoft XT [1,6], CertiKOS [3,4], and $\mu\text{C}/\text{OS-II}$ [14]. The seL4 team [5] is the first to achieve a fully machine-checked functional correctness proof of a high-performance L4-family microkernel in the interactive theorem prover Isabelle/HOL. The kernel is sequential and comprises 8,700 lines of C and 600 lines of assembler. The Verisoft XT project verifies the Hyper-V hypervisor [6] using VCC [1], an automatic verifier for C developed by Microsoft. The hypervisor integrates a small multi-processor operating system and a virtualization layer, collectively consisting of 100,000 lines of C and 5,000 lines of assembly. However, only 20% of the code is verified [1], leaving uncertainty about the complete verification of the OS kernel. Gu et al. [3] verify the mCertiKOS kernel, which is sequential. By extending the mCertiKOS kernel with various features, the same team presents and verifies the CertiKOS kernel, which supports multi-core concurrency and consists of 6,500 lines of C and assembly. Xu et al. [14] verify key modules of $\mu\text{C}/\text{OS-II}$, which is a commercial preemptive OS kernel designed for single-core environments. The verified modules, including the scheduler, timer interrupt handler, mutexes, and message queues, among others, comprise 3,250 lines of C, constituting roughly 50% of the entire kernel. All the proofs for mCertiKOS, CertiKOS and $\mu\text{C}/\text{OS-II}$ are completed in the interactive theorem prover Rocq. Overall, it is evident that completely verified kernels with large codebases are rare. The reasons can be summarized as follows. On the one hand, automated tools seem insufficient to verify entire kernels, since they are typically based on SMT solvers, which are considered black boxes, making them difficult to debug and limiting the expressiveness. On the other hand, interactive theorem provers offer richer logic language but are inefficient and require significant human effort.

With a codebase of 17,000 lines of C, achieving full formal verification for LiteOS-M poses a significant challenge. To improve verification efficiency, we design a formal verification platform named Smart Verifier, which combines automatic and interactive theorem proving techniques. Our platform consists of three main components: an annotation-based verifier, QCP [12] at the front end; an OS verifier OSVAuto [13] at the back end; and a connector module facilitating seamless integration. For the front end, we employ QCP [12], offering a

symbolic executor to automatically generate separation logic verification conditions from annotated programs, which are then proved manually by users in Rocq. For the back end, we use OSVAuto [13], a tool tailored for OS verification, which leverages the state-of-the-art SMT solver Z3 [2] for automatic solving of pure propositions. Furthermore, we develop a connector module that translates propositional formulas purified by QCP into a format compatible with OSVAuto, effectively bridging QCP and OSVAuto.

Similar to prior works [4,14], we verify the functional correctness of LiteOS-M through proving *refinement* relationships, which assert that the low-level concrete implementation strictly follows the high-level abstract specification of the kernel’s functionality. We tailor two verification methods to be compatible with our verification platform. First, it is known that refinement relationships are commonly established through relational Hoare logic quadruples. To facilitate the use of annotation-based verifiers, we encode relational Hoare logic quadruples into standard Hoare logic triples by the method proposed in [11]. Second, inspired by the refinement verification of concurrent programs [8,9], we use state transition systems (STs) to model concurrency at both high and low levels. These verification methods have been implemented and are entirely compatible with our verification platform.

Our verification framework, comprising the platform and the tailored methods, enables us to successfully complete the functional correctness verification of LiteOS-M. The whole verification process takes 12 person-years. During the code review and verification, we discover a total of 17 bugs in LiteOS-M, including concurrency issues, memory errors, numerical overflows, and functional defects. The verification process also contributes to optimize the verification platform. In the scenario of LiteOS-M verification, we propose several new features for QCP, such as support for multi-branch structures and multi-spec derivation, and for OSVAuto, such as support for nested sequences, thereby facilitating the verification process. We also identify and fix numerous bugs in these tools.

This paper is organized as follows. The preliminaries are provided in Section 2, including an introduction to the verification tools QCP and OSVAuto. Section 3 introduces the LiteOS-M kernel and presents an overview of our verification framework. This framework consists of the verification platform and the verification methods, which are detailed in Section 4 and Section 5, respectively. Using the framework, we complete the verification of LiteOS-M and summarize the verification efforts and results in Section 6. Section 7 concludes this paper.

2 Preliminaries

This section introduces the verification tools QCP [12] and OSVAuto [13], both of which are utilized in our verification platform.

2.1 QCP

```

1 struct list {int head; struct list *tail;};
2 struct list *reverse (struct list *p)
3 /*@ With    (l: list Z),
4     Require sll(p,l)
5     Ensure  sll(__return, rev(l)) */
6 {
7     struct list *w, *t, *v;
8     w = (void *) 0; v = p;
9     /*@ Inv exists l1 l2,
10        l == app(rev(l1), l2) && sll(w, l1) * sll(v, l2) */
11     while (v) {
12         t = v->tail; v->tail = w; w = v; v = t; }
13     return w;
14 }

```

Fig. 1: An Annotated Program for Verifying Linked-List Reversal in QCP. (&&: logical conjunction; *: separating conjunction; $sll(p, l)$: a separation logic predicate, stating that address p stores a singly linked list whose elements are given by l . We define sll , app and rev in Rocq; QCP only introduces their declarations.)

QCP [12], short for Qualified C Programming Verifier, is an annotation-based verifier for C programs based on separation logic. Figure 6a illustrates the framework of QCP. Given an annotated C program, the symbolic executor in QCP generates verification conditions automatically, expressed as separation logic entailments and thus referred to as *separation logic verification conditions* (SL VCs). Guided by user-defined, soundness-guaranteed *strategies* [10], these SL VCs undergo a purification phase that removes all spatial formulas and reduces them to pure propositions, which we refer to as *propositional verification conditions* (PROP VCs). Theoretically, QCP can purify any SL VC into a PROP VC, provided that users supply enough strategies. Each resulting PROP VC is handled by a lightweight SMT solver, with support for the theories of uninterpreted functions and linear integer arithmetic. When solving a PROP VC, the SMT solver returns “valid”, “invalid”, or “unknown”, and QCP accordingly either generates a proof automatically, reports an error for user correction, or requires users to manually construct a proof for the original SL VC, as purification may remove essential pure facts.

Example 1. Figure 1 presents an implementation of an in-place linked list reversal and its annotations. The annotations on Lines 3–5 describe the specification this function should satisfy: for any list of integers l , if l is stored in a linked list and this linked list’s head pointer is passed to the function by the program variable p , then the function reverses the linked list and returns the new head pointer $__return$. The assertion on Lines 8–9 describes a loop invariant, that the program state should satisfy every time the program enters the loop body.

2.2 OSVAuto

OSVAuto [13] is designed to automatically prove pure propositions without spatial assertions—that commonly arise when verifying OS kernels.

```

1 query sched_status {
2     fixes tcb : TaskControlBlock;
3     assumes vaild_usedTask_status(tcb.status);
4     assumes status_delay_or_pendtime(tcb.status);
5     shows tcb.status & (STATUS_DELAY | STATUS_PEND) != 0
6     proof { auto }
7 }

```

Fig. 2: A Query in OSVAuto.

OSVAuto adopts a functional language for describing data structures and expressing verification conditions. On the one hand, it provides native support for defining common data structures in OS kernels, such as structures, enumerations, sequences, and maps. On the other hand, the language allows users to express verification conditions in the form of *queries* [13], each of which consists of a list of type parameters, variables, assumptions, a conclusion, and an automatic proof. An example of such a query is shown in Figure 2.

OSVAuto leverages an automation algorithm to prove queries. It employs Z3 [2] as its backend, a state-of-the-art SMT solver that supports a wide range of theories—including arithmetic, arrays, bit-vectors, and uninterpreted functions—and delivers high performance in solving complex constraints.

3 Background and Overview of Our Work

In this section, we introduce the LiteOS-M kernel in Section 3.1 and provide an overview of the verification framework in Section 3.2.

3.1 LiteOS-M

LiteOS-M is a basic kernel of the OpenHarmony operating system, designed for resource-constrained environments with a focus on small footprint, low power consumption, and high performance. Within OpenHarmony, LiteOS-M plays a central role in maintaining system integrity and managing system resources.

The upper part of Figure 3 shows the architecture of the LiteOS-M kernel, which includes subsystems such as task management, software timer, scheduler, tick management, inter-process communication, and memory management. LiteOS-M achieves concurrency in a single-core environment by employing pre-emptive, priority-based scheduling. Within the kernel, execution of a task can be interrupted at any program point (unless interrupts are disabled). Then the control is switched to the interrupt handler. When the handler finishes, it may invoke the scheduler and switch the execution context to the highest priority task, instead of returning to the original interrupted task.

3.2 Overview of the Verification Framework

OS Correctness The *functional correctness* of an OS kernel can be described by *refinement* [5,14], which establishes a correspondence between the concrete implementations of application programming interfaces (APIs) and their abstract

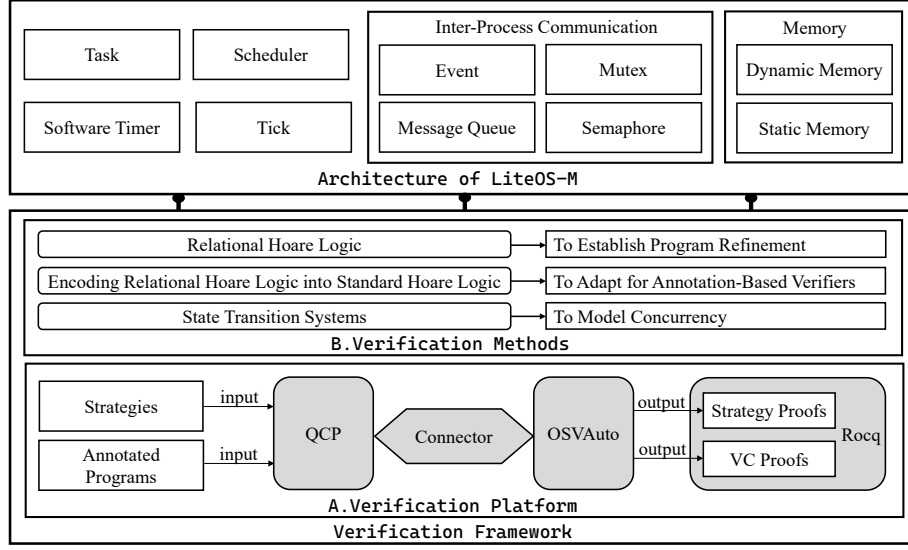


Fig. 3: Verification Framework for LiteOS-M.

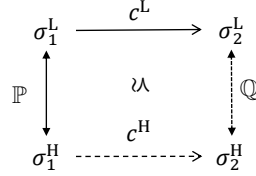


Fig. 4: Program Refinement.

models. Figure 4 presents the refinement relationship between the concrete low-level program c^L and the abstract high-level program c^H . This refinement can be formalized as a relational Hoare quadruple $\{\mathbb{P}\} c^L \lesssim c^H \{Q\}$, meaning that given any initial state pair (σ_1^L, σ_1^H) satisfying $\mathbb{P}$, for any final state σ_2^L after the execution of c^L , there exists a multi-step transition from (c^H, σ_1^H) to $(\text{skip}, \sigma_2^H)$ such that (σ_2^L, σ_2^H) satisfies Q . Here, σ_1^L and σ_2^L are low-level program states, σ_1^H and σ_2^H are high-level program states, and skip represents a no-op high-level program.

The Whole Verification Framework The lower part of Figure 3 presents the verification framework for LiteOS-M. Block A presents the verification platform, Smart Verifier (see Section 4 for details). It consists of QCP, a connector module, and OSVAuto. Block B shows the verification methods (see Section 5 for details). On the one hand, we employ relational Hoare logic to establish program refinement and encode it into standard Hoare logic to facilitate the use of annotation-based verifiers. On the other hand, we utilize STSs to model concurrent behaviors.

A Running Example Figure 5 presents an API in LiteOS-M, which is designed to retrieve the name of the currently running task, with its annotations. In order

```

1 char *LOS_CurTaskNameGet ()
2 /*@
3 With (sg: GV) (st: ST) X
4 Require
5     g_taskCBArray == sg.g_taskCBArray &&
6     sg.g_taskCBArray > 0 &&
7     Exec(LastSeen(st), abs_curTaskNameGet, X) &&
8     OutsideCritical(sg, st)
9 Ensure
10     exists st2,
11         __return != 0 &&
12         Exec(LastSeen(st2), return(__return), X) &&
13         OutsideCritical(sg, st2)
14 */
15 { char *taskName = (char *) 0;
16     /*@ ... */
17     enter_critical();
18     if (g_losTask.runTask != (LosTaskCB *) 0)
19         { taskName = g_losTask.runTask -> taskName; }
20     /*@ ... */
21     exit_critical();
22     return taskName;
23 }

```

Fig. 5: An Annotated Program for Verifying an API in LiteOS-M. (Due to space limitations, the annotations on Line 16 and Line 20 are omitted.)

to ensure safe access, the task name is read within the critical section, while the value is returned outside the critical section.

We verify this API using Smart Verifier. The annotations on Lines 2–14 describe the specification this function should satisfy. The *With* clause universally quantifies over a collection of global variables *sg*, a single high-level program state *st*, and a collection of sets of high-level program states *X*. The *Require* clause specifies the following precondition: (1) the global variable *g_taskCBArray* in the C code coincides with its counterpart *sg.g_taskCBArray* in the Rocq model, (2) *sg.g_taskCBArray* is greater than zero, which guarantees that the *if*-condition on Line 18 holds, (3) from the initial collection *LastSeen(st)*, applying the high-level program *abs_curTaskNameGet* results in the final collection *X*, and (4) the kernel is outside the critical section before invoking this API. Here, *abs_curTaskNameGet* is the abstraction of the API at the high level, *st* refers to the last-seen high-level state within the critical section during the previous invocation of the API, and *LastSeen(st)* represents a collection containing precisely one set, which is the set of reachable high-level states after exiting the critical section, resulting from the nondeterminism in concurrent execution. After the completion of this API call, the post-condition specified in the *Ensure* clause should be satisfied: there exists a single high-level state *st2*, such that (1) the return value of the API is nonzero, (2) from the initial collection *LastSeen(lvs2)*, applying the high-level program

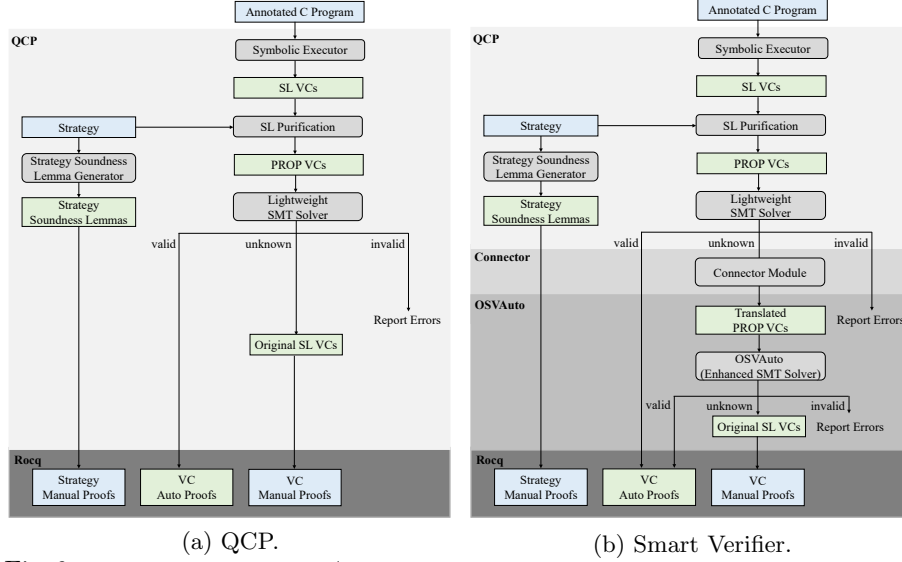


Fig. 6: Verification Platform. (Blue rectangles: files manually completed by users. Green rectangles: files automatically generated by tools. Gray rounded rectangles: tool components.)

`return(__return)` results in the same final collection X , and (3) the kernel is outside the critical section after the API invocation. Here, the high-level program `return(__return)` abstracts the code on Line 22, and `st2` refers to the last-seen high-level state within the critical section during this API call.

The formal definitions of `Exec`, `LastSeen` and `OutsideCritical` will be given in Section 5, in Definitions 1–3, respectively.

4 Verification Platform

In this section, we provide a detailed description of the verification platform developed and utilized in our framework.

Figure 6b presents our verification platform Smart Verifier. When the lightweight SMT solver in QCP returns “unknown” for some PROP VC, the verification platform submits the PROP VC to OSVAuto for further solving. Since OSVAuto adopts a different language for expressing PROP VCs than QCP, we develop a connector module that translates the PROP VC into a query format compatible with OSVAuto. After appropriate preprocessing, the query is passed to Z3, which offers significantly stronger solving capabilities than the lightweight SMT solver in QCP. The subsequent steps after Z3 solving are identical to those following the lightweight SMT solver in QCP.

The implementation of the connector module is described in detail below. In QCP, a PROP VC is constructed from its abstract syntax tree (AST). The connector module traverses the AST and converts each node into its corresponding expression in the language supported by OSVAuto.

```

1 PROP[ valid_usedTask_status(status(tcb));
2     status_delay_or_pendtime(status(tcb)) ]
3 |- PROP[ status(tcb) & (STATUS_DELAY | STATUS_PEND) != 0 ]
    
```

Fig. 7: A PROP VC in QCP.

Example 2. When using QCP alone to verify the scheduler module of LiteOS-M, we encounter a PROP VC (as shown in Figure 7) that cannot be automatically solved by QCP’s built-in SMT solver. This PROP VC states that for any task control block (TCB), if the status of the TCB is valid and is either delayed or timed pending, then the status must be either delayed or pending. The connector module in Smart Verifier reformulates this PROP VC as a query (as shown in Figure 2), which is successfully solved by OSVAuto.

5 Verification Methods

In this section, we present the tailored verification methods in our framework. Section 5.1 establishes program refinement, which is expressed as standard Hoare triples. Section 5.2 models concurrency at both the low level and the high level through STSs. By these verification methods, we can annotate concurrent programs in LiteOS-M, and verify them using Smart Verifier.

5.1 Establishing Program Refinement

Let Prog^L and Prog^H be the set of low-level programs and the set of high-level programs, respectively. Let Σ^L and Σ^H be the set of low-level program states and the set of high-level program states, respectively. Recalling Section 3.2, we use a relational Hoare quadruple $\{\mathbb{P}\} c^L \lesssim c^H \{\mathbb{Q}\}$ to establish program refinement, where $c^L \in \text{Prog}^L$, $c^H \in \text{Prog}^H$ and $\mathbb{P}, \mathbb{Q} \subseteq \Sigma^L \times \Sigma^H$.

To adapt for annotation-based verifiers, we review how to encode relational Hoare quadruples as standard Hoare triples. In real program verification, a binary assertion $\mathbb{P} \subseteq \Sigma^L \times \Sigma^H$ is usually in a form as $\exists \vec{a}. B(\vec{a}) \wedge [P^L(\vec{a})] \wedge [P^H(\vec{a})]$, where $\vec{a}$ represents a list of logical variables, B is a pure proposition, $P^L \subseteq \Sigma^L$, and $P^H \subseteq \Sigma^H$. The unary assertions P^L and P^H can be lifted to the binary assertions $[P^L]$ and $[P^H]$, respectively, in the following manner: for any state pair $(\sigma^L, \sigma^H) \in \Sigma^L \times \Sigma^H$, $(\sigma^L, \sigma^H) \models [P^L]$ holds if and only if (iff) $\sigma^L \models P^L$, and $(\sigma^L, \sigma^H) \models [P^H]$ holds iff $\sigma^H \models P^H$. Building on this, Theorem 1 presents an encoding method, which relies on the following definition.

Definition 1 (Execution Condition [11, Definition 18]). For any $P^H \subseteq \Sigma^H$, $c^H \in \text{Prog}^H$ and $X \subseteq \Sigma^H$, the execution condition is defined as follows:

$$\text{Exec}(P^H, c^H, X) = \exists \sigma^H. (\sigma^H \models P^H) \wedge (\forall \sigma. ((\sigma^H, \sigma) \in \llbracket c^H \rrbracket_{\text{nrn}} \Rightarrow \sigma \models X),$$

where $\llbracket \cdot \rrbracket_{\text{nrn}}$ denotes the denotational semantics.

Theorem 1 (Encoding Relational Hoare Quadruples [11, Theorem 4]).
In order to prove the following program refinement⁵,

$$\begin{aligned} & \left\{ \exists \vec{a}. B_1(\vec{a}) \wedge [P^L(\vec{a})] \wedge [P^H(\vec{a})] \right\} c^L \lesssim c^H \\ & \left\{ \exists \vec{b}. B_2(\vec{b}) \wedge [Q^L(\vec{b})] \wedge [Q^H(\vec{b})] \right\} \end{aligned} \quad (1)$$

it suffices to prove for any $X \subseteq \Sigma^H$, the following standard Hoare triple holds.

$$\begin{aligned} & \left\{ \exists \vec{a}. B_1(\vec{a}) \wedge P^L(\vec{a}) \wedge \text{Exec}(P^H(\vec{a}), c^H, X) \right\} c^L \\ & \left\{ \exists \vec{b}. B_2(\vec{b}) \wedge Q^L(\vec{b}) \wedge \text{Exec}(Q^H(\vec{b}), \text{skip}, X) \right\} \end{aligned} \quad (2)$$

5.2 Modeling Concurrency

In sequential program execution, *determinism* allows the observation of a single program state at any moment. In contrast, concurrent program execution is *nondeterministic*. Although the program resides in a single state at any time, we can only observe a set of reachable program states outside a critical section. Thus, the idea of defining elements in Σ^H (or Σ^L) as observed sets of high-level (or low-level) states is natural. At the high level, we first construct a set ST of single high-level program states, and then Σ^H can be defined as:

$$\Sigma^H = 2^{ST} = \{A \mid A \subseteq ST\}.$$

At the low level, the definition of Σ^L is more intricate and will be provided later.

In order to characterize such observed sets of reachable program states, we define a low-level STS and a high-level STS in Figure 8. Here, the set $GV = GV_1 \times \cdots \times GV_n$, where each GV_i is the set of possible values for global static variable gv_i in the source code.

Remark 1. Actually, the transitions $\rightsquigarrow_{\text{GTrans-H}}$ and $\rightsquigarrow_{\text{GTrans-L}}$ in Figure 8 define the state updates allowed for the current thread within a critical section, and transitions $\rightsquigarrow_{\text{RTrans-H}}$ and $\rightsquigarrow_{\text{RTrans-L}}$ define those allowed for other threads outside a critical section. Thus, we require these transitions satisfy conditions (i) and (ii) in Figure 8. Moreover, we also require that $\rightsquigarrow_{\text{RTrans-L}}$ and $\rightsquigarrow_{\text{RTrans-H}}$ satisfy the reflexivity and the transitivity, corresponding to the case that no other thread updates the state, and the case that the state is updated multiple times by multiple threads, respectively.

Modeling Concurrency at the High Level Note that the program execution within a critical section is sequential, allowing us to obtain a specific program state. To formalize the set of reachable high-level program states outside a critical section, we define the assertion $\text{LastSeen}(st)$, where st refers to the last-seen high-level program state within the critical section.

⁵ The relational Hoare quadruple Eq. (1) can be transformed into an equivalent triple form, as used in [11]. Then, Theorem 1 is a straight conclusion of [11, Theorem 4].

Low-Level STS :	$(GV \times ST, \rightsquigarrow_{GTrans-L}, \rightsquigarrow_{RTrans-L})$
State Space :	$GV \times ST$
High-Level STS :	$(ST, \rightsquigarrow_{GTrans-H}, \rightsquigarrow_{RTrans-H})$
State Space :	ST
Transitions :	$\rightsquigarrow_{GTrans-L}, \rightsquigarrow_{RTrans-L} \subseteq (GV \times ST) \times (GV \times ST),$ $\rightsquigarrow_{GTrans-H}, \rightsquigarrow_{RTrans-H} \subseteq ST \times ST$ such that (i) $(sg, st) \rightsquigarrow_{GTrans-L} (sg', st')$ iff $sg = sg'$ and $st \rightsquigarrow_{GTrans-H} st'$; (ii) $(sg, st) \rightsquigarrow_{RTrans-L} (sg', st')$ iff $sg = sg'$ and $st \rightsquigarrow_{RTrans-H} st'$; (iii) $\rightsquigarrow_{RTrans-L}, \rightsquigarrow_{RTrans-H}$ satisfy the reflexivity and the transitivity

Fig. 8: The Low-Level STS and the High-Level STS.

Definition 2 (Assertion LastSeen). *Given any $st \in ST$, we define the high-level assertion $\text{LastSeen}(st) \subseteq \Sigma^H$ as follows:*

$$\sigma^H \models \text{LastSeen}(st) \text{ iff } \sigma^H = \{st' \in ST \mid st \rightsquigarrow_{RTrans-H} st'\}.$$

For an API c^L , it is generally the case that the kernel is outside a critical section before and after invoking the API. Thus, when no additional constraints are required, the high-level assertions P^H, Q^H in Eq. (2) can be expressed with the predicate **LastSeen**. Specifically, Theorem 1 becomes that, it suffices to prove for any $X \subseteq \Sigma^H$ and any $st \in ST$, the following standard Hoare triple holds.

$$\left\{ \exists \vec{a}. B_1(\vec{a}) \wedge P^L(\vec{a}) \wedge \text{Exec}(\text{LastSeen}(st), c^H, X) \right\} c^L \\ \left\{ \exists \vec{b}, st_2. B_2(\vec{b}) \wedge Q^L(\vec{b}) \wedge \text{Exec}(\text{LastSeen}(st_2), \text{skip}, X) \right\}$$

Here, $st \in ST$ represents the last-seen high-level program state within the critical section during the previous invocation of c^L , and $st_2 \in ST$ represents the one during the current invocation.

Modeling Concurrency at the Low Level It remains to define the set of low-level program states Σ^L and to specify the standard Hoare triples for the statements “`enter_critical()`” and “`exit_critical()`” (Lines 17 and 21 in Figure 5), which mark the entry to and the exit from a critical section, respectively.

We first refine the state space $GV \times ST$ of the low-level STS into the following set CST , distinguishing the states inside and outside a critical section.

$$CST = \{\text{inside}(sg, st), \text{outside}(sg, st) \mid (sg, st) \in GV \times ST\}$$

We also define a function $\mathcal{T}$ such that for any $(sg, st) \in GV \times ST$,

$$\mathcal{T}(\text{inside}(sg, st)) = \{\text{tt}\}, \mathcal{T}(\text{outside}(sg, st)) = \emptyset,$$

which assigns to each element of CST the set of tokens it owns. The set of low-level program states Σ^L is defined by

$$\Sigma^L = \{(\mathcal{M}, s_{state}, s_{token}) \mid \mathcal{M} : \text{Addr} \rightarrow \text{Mem}, s_{state} \subseteq CST, s_{token} \subseteq \{\text{tt}\}\},$$

where $\mathcal{M}$ is a memory mapping (assigning each address to a memory state, e.g., no permission, uninitialized, or a concrete value), s_{state} is a set of reachable states, and s_{token} is a set of held tokens.

$\{ \text{OutsideCritical}(sg, st) \}$	$\{ \text{InsideCritical}(sg_1, st_1) **$
$\text{enter_critical}();$	$\text{OsInv}(sg_2, st_2) \ \&\& \ st_2 = \text{Replace}(st_1) \ \&\&$
$\{ \exists sg_1, st_1. (sg, st) \rightsquigarrow_{\text{RTrans-L}} (sg_1, st_1) \ \&\&$	$(sg_1, st_1) \rightsquigarrow_{\text{GTrans-L}} (sg_2, st_2) \}$
$\text{InsideCritical}(sg_1, st_1) **$	$\text{exit_critical}();$
$\text{OsInv}(sg_1, st_1) \}$	$\{ \text{OutsideCritical}(sg_2, st_2) \}$

Fig. 9: Standard Hoare Triples for the Entry to and the Exit from a Critical Section. (Here, $\text{OsInv}(sg, st)$ is an invariant that must be satisfied by the shared resources when entering and exiting a critical section; $\text{Replace}(st_1)$ is a function updating the high-level program state induced by executing the code within the critical section.)

Definition 3 (Assertions `InsideCritical` and `OutsideCritical`). *Given any $(sg, st) \in GV \times ST$, we define the low-level assertions $\text{InsideCritical}(sg, st)$, $\text{OutsideCritical}(sg, st) \subseteq \Sigma^L$ as follows:*

$$\begin{aligned} \sigma^L \models \text{InsideCritical}(sg, st) &\text{ iff } \sigma^L = (\mathcal{M}_0, \{\text{inside}(sg, st)\}, \{\text{tt}\}), \\ \sigma^L \models \text{OutsideCritical}(sg, st) &\text{ iff } \sigma^L = (\mathcal{M}_0, \{\text{outside}(sg', st') \mid (sg, st) \rightsquigarrow_{\text{RTrans-L}} (sg', st')\}, \emptyset), \end{aligned}$$

where $\mathcal{M}_0$ maps every address to the “no permission” memory state.

Remark 2. In assertions $\text{InsideCritical}(sg, st)$ and $\text{OutsideCritical}(sg, st)$, st refers to the last-seen high-level program state within a critical section. It is important to note that these two assertions are used to identify low-level program states inside and outside a critical section, without modifying the memory; while any other low-level assertion only cares about the memory, that is, specifying an element in the form of $(\mathcal{M}, CST, \emptyset)$. And, we define the separating conjunction of two low-level assertions as $\{(\mathcal{M}, s_{state}, s_{token})\} ** \{(\mathcal{M}', s'_{state}, s'_{token})\} = \{(\mathcal{M} * \mathcal{M}', s_{state} \cap s'_{state}, s_{token} \cup s'_{token})\}$, where $*$ denotes the standard separating conjunction for memory mappings.

Finally, we present standard Hoare triples for “`enter_critical();`” and “`exit_critical();`” in Figure 9. Every entry to and exit from a critical section requires the accessed shared resources to satisfy a specific invariant.

Remark 3. Inspired by [8,9], we propose an STS-based framework to model concurrency in Section 5.2. This basic framework can be tailored by readers to meet specific verification demands. For example, we only present the simplest case with a single token `tt`, distinguishing the states inside the critical section from those outside. For more complex scenarios, such as nested critical sections, the token sort can be set as the set of integers: a state outside the critical section has a token value of 0, while a state inside the critical section has a token value equal to its nesting depth.

```

1 Definition liteos_inv (sg: GV) (st: ST):
2   Assertion :=
3     task_inv sg st.(taskst) **
4     sched_inv sg st.(schedst) ** ....
5
6 Definition task_inv (sg: GV) (s: TaskST):
7   Assertion :=
8     [| task_vaild_cond_prop sg s |] &&
9     store_used_task sg s.(usedTaskList) **
10    store_free_task sg s.(freeTaskList) **
11    store_recycle_task sg s.(recycleTaskList) ** ....

```

Fig. 10: The Invariant for LiteOS-M. (Here, `liteos_inv` is the invariant for LiteOS-M, conjuncting the invariants of all kernel modules, such as `task_inv` for the task module; `ST` defines the type of single high-level program states for LiteOS-M, comprising the types for all kernel modules, such as `TaskST` for the task module.)

6 Verification of LiteOS-M

Using the above framework, we successfully complete the verification of LiteOS-M. Section 6.1 constructs the invariant for LiteOS-M. Section 6.2 evaluates the Smart Verifier platform. Finally, Section 6.3 presents the verification efforts and identified defects.

6.1 The Invariant for LiteOS-M

As shown in Figure 10, we construct an invariant `liteos_inv` for the LiteOS-M kernel, which conjuncts the invariants of all kernel modules.

Take the task module invariant `task_inv` as an example (also see Figure 10). The pure proposition on Line 8 characterizes certain properties that a task maintains when the kernel enters and exits a critical section, such as (1) each task’s ID and priority lie within the allowed range; (2) each task must be in one of these five states: running, ready, pending, delayed, or suspended; and (3) each task must appear in one of these three lists: the used task list, the free task list, or the recycled task list. The separation logic assertion on Line 9 states that the used task list whose elements are given by `s.(usedTaskList)` is stored in the corresponding global address variable. The separation logic assertions on Lines 10–11 are similar, describing the storage of the free task list and the recycled task list, respectively.

6.2 Evaluation of Smart Verifier

Most current OS verification projects are conducted using interactive theorem provers, such as `seL4` [5] in Isabelle/HOL and `μC/OS-II` [14] in Rocq, resulting in little to no automation. We use Smart Verifier to complete the verification of LiteOS-M. On the one hand, given annotated C programs, Smart Verifier can automatically generate verification conditions. On the other hand, Smart Verifier can automatically prove some verification conditions by built-in SMT

Table 1: Automation of Smart Verifier. (# **VC**: Number of generated verification conditions (VCs). # **Manual VC**: Number of VCs manually proved in Rocq. # **Auto VC**: Number of VCs automatically proved by SMT solvers.)

	Smart Verifier			
	# VC	# Manual VC	# Auto VC	# Auto VC : # VC
Task	1840	320	1520	83%
Scheduler	836	124	712	85%
Dynamic Memory	1473	240	1233	84%
Total	4149	684	3465	84%

solvers. Taking the task, scheduler, and dynamic memory modules of LiteOS-M as examples, Smart Verifier automatically proves exceeding 80% of all the generated verification conditions (see Table 1).

6.3 Verification Efforts and Identified Defects

Verification Efforts We have verified 12 key modules of LiteOS-M⁶, including task, scheduler, message queue, mutex, semaphore, event, sorted linked list, software timer, tick, static memory and dynamic memory. To enhance access control capabilities, an access control module has been incorporated into LiteOS-M, and we have verified this module as well. These verified modules comprise around 17,000 lines of C code including header files, comments, and blank lines in their original format, which we refer to as *raw code*. After excluding header files, comments, auxiliary functions used for debugging (such as printing functions), and blank lines, the corresponding code amounts to around 5,400 lines, which we refer to as *clean code*.

Table 2 summarizes the size of each verified module, along with a detailed breakdown of the code size for each part in the verification. In addition to the libraries for each module, the verification relies on a *basic library* that includes the global variables, the state transition systems, as well as the invariant and the high-level program state defined for LiteOS-M. The basic library contains 1,046 lines of Rocq code. Table 3 compares the verification of LiteOS-M with two other verified operating systems, namely seL4 [5] and μ C/OS-II [14]. Thanks to Smart Verifier and the tailored verification methods, LiteOS-M achieves a ratio of verification-related code to raw code at around 8 : 1.

Identified Defects During the code review and verification, we found in total 17 critical bugs and 3 additional defects, all of which have been confirmed and fixed by the developers. The 17 critical bugs include: 10 concurrency bugs, of which 8 are race conditions caused by missing critical section protection; 3 numerical overflow errors, all resulting from integer overflows; 3 functional errors, where the corresponding functions fail to implement the intended behavior; and 1 memory error, which is a use-after-free vulnerability. The 3 additional defects are attributed to the setting of an unused lock variable, the passing of unused function parameters, and a flaw in linked list traversal. Furthermore, the two

⁶ https://gitee.com/openharmony/kernel_liteos_m/tree/OpenHarmony-v4.0-Release

Table 2: Verified LiteOS-M. (**C code**: lines of source C code; **Raw**: code in the original format; **Clean**: code excluding header files, comments, auxiliary functions, and blank lines. **Library**: lines of Rocq code for the kernel’s library, including predicates, functions, and invariants. **Strategy**: lines of strategy code. **Annotation**: lines of annotation code, excluding source C code. **Proof**: lines of Rocq code for manual proofs; **Strategy**: proofs for strategies; **Manual VC**: proofs for manual VCs.)

Verified Module	C Code		Library	Strategy	Annotation	Proof	
	Raw	Clean				Strategy	Manual VC
Task	3722	730	3253	2242	5176	1598	10567
Scheduler	790	509	2042	328	2018	269	7283
Message Queue	2537	747	2137	1948	4473	3671	8254
Mutex	1022	459	2681	643	3463	622	10194
Semaphore	937	338	2735	472	2565	463	6712
Event	782	332	4720	167	1580	188	6342
Sorted Linked List	300	211	1056	237	823	248	3168
Software Timer	1491	620	2416	1232	2708	1062	8998
Tick	1129	244	410	98	485	103	1572
Static Memory	567	174	271	206	326	156	775
Dynamic Memory	3339	827	1430	465	2973	382	10623
Access Control	512	245	1579	696	1062	618	3139
Total	17128	5436	24754	8525	26023	9382	77783

Table 3: Comparison of OS Kernel Verification. (**Raw Code**: lines of source code in the original format. **Verification Code**: lines of verification-related code.)

	Concurrency	Raw Code	Verification Code	Verification Code : Raw Code	Verification Efforts (person-years)
seL4 [5]	not supported	9300	200000	21: 1	20
μ C/OS-II [14]	single-core concurrency	3250	104808	32: 1	5.5
LiteOS-M	single-core concurrency	17128	147513	8: 1	12

critical bugs we identified are confirmed to exist in the OpenHarmony LiteOS-A kernel as well.

7 Conclusion

We design a formal verification platform and tailor refinement-based verification methods to ensure the functional correctness of LiteOS-M. It is worth noting that although our verification framework is developed to verify LiteOS-M, it is a general verification framework and can be reused to verify other preemptive OS kernels implemented in C.

Our experience verifying LiteOS-M suggests that verifying an existing OS kernel is more challenging than developing one for verification purposes. A considerable amount of time is spent on understanding the source code, identifying defects, and re-verifying after code modifications. Ideally, the development and formal verification of an OS kernel should be conducted simultaneously. Additionally, even with individual module responsibilities, unifying specifications and invariants is time-consuming at the code merging stage. This motivates us to explore more efficient collaborative ways for OS kernel verification tasks.

References

1. Cohen, E., Dahlweid, M., Hillebrand, M., Leinenbach, D., Moskal, M., Santen, T., Schulte, W., Tobies, S.: Vcc: A practical system for verifying concurrent c. In: International Conference on Theorem Proving in Higher Order Logics. pp. 23–42. Springer (2009)
2. De Moura, L., Bjørner, N.: Z3: An efficient smt solver. In: International conference on Tools and Algorithms for the Construction and Analysis of Systems. pp. 337–340. Springer (2008)
3. Gu, R., Koenig, J., Ramananandro, T., Shao, Z., Wu, X., Weng, S.C., Zhang, H., Guo, Y.: Deep specifications and certified abstraction layers. *ACM SIGPLAN Notices* **50**(1), 595–608 (2015)
4. Gu, R., Shao, Z., Chen, H., Wu, X.N., Kim, J., Sjöberg, V., Costanzo, D.: CertiKOS: An extensible architecture for building certified concurrent OS kernels. In: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16). pp. 653–669 (2016)
5. Klein, G., Elphinstone, K., Heiser, G., Andronick, J., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., et al.: sel4: Formal verification of an os kernel. In: Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles. pp. 207–220 (2009)
6. Leinenbach, D., Santen, T.: Verifying the microsoft hyper-v hypervisor with vcc. In: International Symposium on Formal Methods. pp. 806–809. Springer (2009)
7. OpenAtom Foundation: OpenHarmony Official Website. <https://www.openharmony.cn> (2020)
8. Turon, A., Dreyer, D., Birkedal, L.: Unifying refinement and hoare-style reasoning in a logic for higher-order concurrency. In: Proceedings of the 18th ACM SIGPLAN international conference on Functional programming. pp. 377–390 (2013)
9. Turon, A.J., Thamsborg, J., Ahmed, A., Birkedal, L., Dreyer, D.: Logical relations for fine-grained concurrency. *Acm Sigplan Notices* **48**(1), 343–356 (2013)
10. Wang, Z., Wu, X., Fang, Y., Li, C., Zhong, H., Xie, L., Cao, Q., Hu, Z.: Stelis: A strategy language for purifying separation logic entailments. *arXiv preprint arXiv:2512.05159* (2025)
11. Wu, S., Wu, X., Cao, Q.: Encode the forall exists relational hoare logic into standard hoare logic. *Proceedings of the ACM on Programming Languages* **9**(OOPSLA2), 2454–2481 (2025)
12. Wu, X., Feng, Y., Lu, X., Lin, T., Liu, K., Wang, Z., Wu, S., Xie, L., Yang, C., Zhong, H., et al.: Neat: Qcp: A practical separation logic-based c program verification tool. *arXiv preprint arXiv:2505.12878* (2025)
13. Wu, Y., Xia, B., Xu, J., Zhan, B., Zhao, T.: Osvauto: automatic proofs about functional specifications in os verification. *arXiv preprint arXiv:2403.13457* (2024)
14. Xu, F., Fu, M., Feng, X., Zhang, X., Zhang, H., Li, Z.: A practical verification framework for preemptive os kernels. In: International Conference on Computer Aided Verification. pp. 59–79. Springer (2016)